

编程作业-第八章

作业说明

本次作业共包含一道大题两道小题。我们给同学们提供了代码与测评框架。下载后的文件树应如下所示：

```
hw8.zip
├── t1.py
├── t2.py
├── ticket_pool.py
├── test.py
└── hw8.pdf
```

其中`ticket_pool.py`和`test.py`分别是本次作业的部分代码框架与测评框架。你仅需阅读这两个文件，但**不可以修改**。你需要结合注释信息来补全`t1.py`和`t2.py`的代码，需要修改的部分已经打上TODO标签。

补全代码后，将`t1.py`和`t2.py`打包成zip文件，命名为{你的名字}_{你的学号}.zip，**并提交至网络学堂**。注意你仅需提供这两个文件，无需提交`ticket_pool.py`和`test.py`。

```
例如 张三_2025010001.zip
├── t1.py
└── t2.py
```

请注意：

- 编程作业为个人作业，请独立完成。如果提交的代码参考了资料或个人，请在源代码文件中通过注释来标注来源，否则视为抄袭。
- 为方便评判，请同学们按照规定格式提交作业。

题目：多线程抢票退票系统(50分+50分)

本题需要大家运用多线程编程的知识，维护一个多线程抢票退票系统。

代码框架介绍

`ticket_pool.py`实现了辅助类TicketPool，用于模拟票池。其接口包括获取剩余票数、抢票接口、退票接口。大家可以结合注释来理解。

`test.py`是本次作业的测评框架。命令行输入`python test.py`即可运行。如果你正确实现了两个小题的所有代码，预期输出如下所示：

```
=== Testing Problem 1 ===
Case (30, 1): Accept
Case (100, 5): Accept
Case (300, 10): Accept
Case (500, 30): Accept
Case (3000, 100): Accept
=== Testing Problem 2 ===
Case (10, 1, 1): Accept
```

```
Case (0, 2, 1): Accept
Case (0, 5, 3): Accept
Case (5, 4, 2): Accept
Case (10, 10, 5): Accept

=====
All tests passed!
```

如果哪一个测例挂了，你也可以通过输出信息来精确定位。本次作业我们公开了测例，但你**不可以修改测例**，**也不可以对测例进行硬编码**。测评时我们也只会运行`test.py`，每个测例占十分。只要你的输出跟上面的一致，就可以获得全部分数。

第一小题：抢票(50分)

这一小题要求大家简单模拟并发情况下的抢票操作。

在`t1.py`中定义了继承于`threading.Thread`的`Ticket`类，用于模拟单个客户的抢票操作。

每个用户(线程)只会进行一次抢票操作，具体逻辑在`run`函数中。首先需要调用`TicketPool.get_rest`来查询票池中剩余的票数。如果剩余票数不少于要抢的票数，则在票池中抢走需要的票数；如果剩余票数小于需要的票数，则在票池中抢走剩余的票数。可以调用`TicketPool.reduce_rest`进行抢票操作。请注意`TicketPool`对象只有一个，而`Ticket`对象有多个，抢票的多个线程是并发的，程序需要利用锁来处理冲突。

正确编写完这部分代码后，你可以运行`test.py`进行测试。此时Testing Problem 1下面的五个测例应该全部输出Accept。

第二小题：退票 + 条件等待抢票(50分)

在真实票务系统中，还可能有人退票。如果某一时刻抢票者发现余票不足，但等到另一用户退票时立刻重新抢票，则有很大概率可以抢票成功。这一小题要求大家使用条件变量`Condition`来模拟这一过程。

我们已经在`TicketPool`类中定义好了条件变量。请通过`TicketPool.get_condition`方法获取与该票池绑定的`Condition`对象，不要自行创建新的`Condition`。

在`t2.py`中定义了继承于`threading.Thread`的`TicketBuyer`类和`TicketRefunder`类，分别用于模拟抢票和退票的用户。所有对票池的访问必须在`Condition`的锁保护下完成。

与第一小题不同，本小题中抢票者“不抢完票不罢休”，一定要**累计**抢到对应数量的票才会结束线程。我们的测例也保证每个抢票者最终都能抢够所需票数并退出。

若当前余票为0，则抢票者会陷入阻塞等待，你可以使用`Condition.wait()`来实现这一点。若当前存在余票，则抢票者会尝试抢走`min(剩余票数, 还需抢票的数量)`张票。在抢票者成功抢到票、或者退票者成功退完票后，需唤醒正在等待的其他抢票者(线程)，你可以使用`Condition.notify_all()`来实现这一点。

你可以调用`TicketPool.add_rest`进行退票操作。

示例: 假设某抢票者需要100张票，而此时票池里只剩90张，则他会先抢走这90张票并进入等待。之后某一退票者成功退掉30张票，此时退票者会唤醒其余所有在等待的抢票者。此时假设原抢票者获取了锁，则他会继续抢走10张票（因为之前已经抢了90张，仅需要再抢10张）。此时票池中还剩下20张余票。

正确编写完这部分代码后，你可以运行`test.py`进行测试。此时Testing Problem 2下面的五个测例应该全部输出Accept。